

Solutions: Assembly Language Programming

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

Overview

The following solutions utilise a generic Load-Store ISA (Instruction Set Architecture) similar to RISC-V or MIPS to demonstrate the logic required for each problem[cite: 33].

Problem 1: Basic Arithmetic - Addition

Write an assembly language program to add two 8-bit numbers stored in memory locations A and B. Store the result in memory location C.

Solution:

```
LOAD  R1, A      ; Load value at address A into R1
LOAD  R2, B      ; Load value at address B into R2
ADD   R3, R1, R2 ; R3 = R1 + R2
STORE R3, C      ; Store result in memory location C
```

Problem 2: Basic Arithmetic - Subtraction

Write an assembly language program to subtract one 8-bit number from another. Assume that the value stored in memory location A is greater than the value stored in memory location B. Store the result in C.

Solution:

```
LOAD  R1, A      ; Load value A
LOAD  R2, B      ; Load value B
SUB   R3, R1, R2 ; R3 = R1 - R2
STORE R3, C      ; Store result in C
```

Problem 3: Repeated Addition

Write an assembly language program to multiply two positive integers using *repeated addition*. Do not use a hardware multiply instruction.

Solution:

```
        LOAD  R1, A      ; Multiplicand
        LOAD  R2, B      ; Multiplier (Counter)
        SET   R3, 0      ; Initialize Result = 0
LOOP:   BEQ   R2, 0, END ; If counter is 0, exit
        ADD   R3, R3, R1 ; Result += Multiplicand
        SUB   R2, R2, 1   ; Decrement counter
        JUMP  LOOP
END:    STORE R3, C
```

Problem 4: Loops - Sum of Natural Numbers

Write an assembly language program to compute the sum of the first N natural numbers using a loop. The value of N is stored in memory.

Solution:

```
        LOAD  R1, N      ; Load N
        SET   R2, 0      ; Initialize Sum = 0
LOOP:   BEQ   R1, 0, DONE
        ADD   R2, R2, R1 ; Sum = Sum + N
        SUB   R1, R1, 1   ; N = N - 1
        JUMP  LOOP
DONE:   STORE R2, RESULT
```

Problem 5: Array Processing

Given an array of signed integers stored in memory, write an assembly language program to count how many elements of the array are positive. Store the count in a separate memory location.

Solution:

```
        LOAD  R1, ADDR   ; Base address of array
        LOAD  R2, SIZE   ; Number of elements
        SET   R3, 0      ; Count = 0
LOOP:   BEQ   R2, 0, END
        LOAD  R4, (R1)   ; Load element
        BLEZ  R4, SKIP   ; If element <= 0, skip
        ADD   R3, R3, 1   ; Increment count
SKIP:   ADD   R1, R1, 4   ; Move to next element (4-byte)
        SUB   R2, R2, 1   ; Decrement size
```

```
        JUMP  LOOP
END:    STORE R3, POS_COUNT
```

Problem 6: Conditional Branching

Write an assembly language program using conditional branching to determine whether a given integer is *even* or *odd*. Store the result as a flag in memory.

Solution:

```
        LOAD  R1, NUM
        AND   R2, R1, 1    ; Mask all but LSB
        BEQ   R2, 0, EVEN
        SET   R3, 1        ; 1 for Odd
        JUMP  STORE_IT
EVEN:    SET   R3, 0        ; 0 for Even
STORE_IT: STORE R3, FLAG
```

Problem 7: Logic Operations

Write an assembly language program to find the largest of three numbers stored in memory locations A, B, and C. Store the largest value in MAX.

Solution:

```
        LOAD  R1, A
        LOAD  R2, B
        LOAD  R3, C
        MOV   R4, R1        ; Assume A is MAX
        BGE   R4, R2, CHKC
        MOV   R4, R2        ; B is larger than A
CHKC:    BGE   R4, R3, END
        MOV   R4, R3        ; C is the largest
END:     STORE R4, MAX
```

Problem 8: Iterative Factorial

Write an assembly language program to compute the factorial of a positive integer using an iterative approach.

Solution:

```

        LOAD   R1, N
        SET    R2, 1      ; Result = 1
LOOP:   BLEZ   R1, END
        MUL    R2, R2, R1 ; Result = Result * N
        SUB    R1, R1, 1
        JUMP   LOOP
END:    STORE  R2, FACT

```

Problem 9: Control Flow - Switch Statement

Implement a multi-way branch (switch-case) in assembly. Given an input variable `op_code` in memory, perform the following:

- If `op_code == 1`: Increment the value in register R1.
- If `op_code == 2`: Decrement the value in register R1.
- If `op_code == 3`: Clear (zero out) register R1.
- **Default**: Store a value of 0xFF in memory location `ERROR`.

Solution:

```

        LOAD   R2, op_code
CASE1:  BNE    R2, 1, CASE2
        ADD    R1, R1, 1
        JUMP   DONE
CASE2:  BNE    R2, 2, CASE3
        SUB    R1, R1, 1
        JUMP   DONE
CASE3:  BNE    R2, 3, DEFAULT
        SET    R1, 0
        JUMP   DONE
DEFAULT: SET    R4, 0xFF
        STORE  R4, ERROR
DONE:   ; Exit switch

```

Problem 10: Stack and Subroutines

Write an assembly routine that implements a simple function call. The program should push two arguments onto the stack, call a subroutine to compute their difference, and return the result in a designated register.

Solution:

```
MAIN: LOAD  R1, A
      PUSH  R1          ; Argument 1
      LOAD  R2, B
      PUSH  R2          ; Argument 2
      CALL  DIFF        ; Subroutine call
      STORE R3, RESULT ; R3 holds return value
      HALT

DIFF: POP    R5          ; Pop B
      POP    R4          ; Pop A
      SUB    R3, R4, R5 ; Return A - B
      RET
```